Introduction

A *data descriptor* (or data paper) publishes a dataset as a citable research object. Unlike an analysis paper, its contribution is not a statistical finding but a **contract**: a precise, machine-readable account of what the dataset contains, how it was produced, how it is licensed, and what quality guarantees it carries. The FAIR guiding principles — Findable, Accessible, Interoperable, Reusable (Wilkinson et al. 2016) — frame why this matters: a dataset is only reusable to the extent that its structure and provenance are legible to both humans and machines.

What this template provides I

This exemplar packages the recurring moving parts of a data descriptor so that a fork can start from a working, tested baseline:

What this template provides I

- A **schema / data dictionary**: named fields with declared types, nullability, units, and value constraints (patterns, enumerations, numeric bounds).
- A **file inventory**: each shipped file with its media type, a sha256 checksum, and a row count.
- A **provenance chain**: the ordered steps and agents that produced the release.
- A **license boundary**: an explicit reuse license, and the discipline of publishing *metadata and checksums* rather than bundling restricted bytes.
- A **validation gate**: a tested library that rejects malformed, unsafe, or incomplete descriptors and verifies declared checksums against real bytes.

Scope and honesty I

The shipped dataset is **synthetic and deliberately small** — it exists to make the workflow runnable and testable, not to support any empirical claim. Its values are generated deterministically. Accordingly, this manuscript restricts its claims to dataset *structure, provenance, quality, and release readiness*. When a real dataset is forked in, the same validation and figure code applies unchanged; only the descriptor and fixture files are replaced. The sections that follow describe the demonstration dataset (sec. 3), its schema (sec. 4), its provenance (sec. 5), and the quality gate that binds the descriptor to the bytes it names (sec. 6), and close with usage and forking notes (sec. 7).